

Synnal - Documentação técnica complementar

Documento complementar ao README. O objetivo aqui é registrar apenas decisões internas, fluxo de execução e limitações relevantes para manutenção do projeto.

1. Fluxo interno da aplicação

Flutter UI/BLoC -> Repository -> Client Dart -> MatrixService (Flutter Rust Bridge) -> Matrix Rust SDK -> Synapse. Os repositories convertem os tipos gerados pelo bridge em modelos da aplicação, evitando que a camada de interface dependa diretamente das estruturas Rust.

O **MatrixService** mantém um único **MatrixClient** substituível, protegido por **RwLock**. Operações de autenticação usam **Mutex** para evitar concorrência durante login, restauração, cadastro e logout. O client é recriado antes de um novo login e após logout.

2. Sessão, dispositivo e armazenamento local

- O profile definido por **SYNNAL_PROFILE** isola session, device ID, passphrase e o diretório SQLite do Matrix. Isso permite executar perfis locais independentes sem compartilhar estado.
- Session, device ID e passphrase são mantidos com **flutter_secure_storage**. A passphrase do store é gerada com 32 bytes aleatórios usando **Random.secure** e reutilizada pelo mesmo profile.
- O Matrix Rust SDK usa um store SQLite em **matrix-store-<profile>**. No login, o device ID salvo é reaproveitado quando disponível.
- A restauração de sessão remove a sessão local quando detecta token inválido. O login possui no máximo duas tentativas, intervalo de 500 ms e timeout global de 30 segundos; erros de credencial não são repetidos.

3. Salas e mensagens

- A carga inicial de salas executa um **sync_once** e separa salas aceitas e convites. Em seguida, o acompanhamento contínuo usa **sync_with_callback** com timeout de 30 segundos.
- A criação de sala usa preset privado e exige pelo menos um usuário convidado. A ação de apagar uma sala significa sair dela; para salas já aceitas, o client também executa **forget**.
- O chat carrega até **50 mensagens** por sala, somente eventos **m.text**, e entrega a lista em ordem cronológica.
- Novas mensagens são recebidas por event handler no Rust, deduplicadas por **event_id** e enviadas ao Flutter por **StreamSink**. **RoomsBloc** e **ChatBloc** cancelam suas subscriptions ao serem fechados.

4. Mapa rápido para manutenção

Responsabilidade	Arquivo principal
Bootstrap, profile e criação do MatrixService	lib/main.dart
Navegação conforme estado de autenticação	lib/app/app.dart
Sessão, retry e restauração	lib/features/auth/repositories/auth_repository.dart
Secure storage por profile	lib/features/auth/clients/auth_storage_client.dart
Estado e streaming de salas	lib/features/rooms/bloc/rooms_bloc.dart
Estado e streaming de chat	lib/features/chat/bloc/chat_bloc.dart
API exposta ao Dart	rust/src/api/matrix.rs
Integração efetiva com Matrix SDK	rust/src/api/client.rs

5. Limitações atuais observadas no código

- Não há paginação de histórico além das 50 mensagens carregadas inicialmente.
- A camada de chat trata apenas mensagens de texto; anexos, reações, respostas, recibos de leitura e recursos semelhantes não estão representados.
- Erros vindos do Rust chegam majoritariamente como **String**. Para evolução, erros tipados facilitariam tratamento específico na camada Flutter.
- O homeserver é definido diretamente no bootstrap; uma configuração externa por ambiente reduziria acoplamento entre desenvolvimento e distribuição.
- No estado atual do repositório não existe uma suite de testes em **test/** ou **integration_test/**; há apenas o driver em **test_driver/integration_test.dart**.
- Arquivos gerados pelo **flutter_rust_bridge** em **lib/src/rust/** e **rust/src/frb_generated.rs** devem ser regenerados, e não editados manualmente.